



## *Inteligencia Artificial - GDDVV*

**Profesores:** Luis Peña Sánchez y Nicolás Tapia Sanz

**Curso:** 2024-2025

# Práctica 2. Algoritmos de Búsqueda

## Contenido

|                                          |   |
|------------------------------------------|---|
| Fecha de Propuesta                       | 1 |
| Fecha de Entrega                         | 1 |
| Objetivos y Competencias                 | 1 |
| Grupos                                   | 1 |
| Diseño e Implementación                  | 1 |
| Paquete de definición del escenario      | 2 |
| Descripción detallada del entorno        | 2 |
| Reglas Generales del Escenario           | 3 |
| Asignación de un Controlador             | 3 |
| Objetivos específicos                    | 4 |
| Apartado 1: Búsqueda de camino de salida | 4 |
| Apartado 2: Atrapar a los enemigos       | 4 |
| Análisis necesario                       | 4 |
| Entrega y Documentación                  | 4 |
| Criterios de Evaluación                  | 5 |

## Fecha de Propuesta

18/12/2024

## Fecha de Entrega

07/02/2025 o (examen Extraordinaria TBD)

## Objetivos y Competencias

El objetivo de la práctica es implementar distintos algoritmos de búsqueda dentro de un entorno de Unity3D. La meta final del trabajo consiste en desarrollar un conjunto de clases de C# que implementen los algoritmos que permitan resolver los distintos problemas que plantearemos dentro del escenario a desarrollar.



## Grupos

La entrega se realizará en grupos de hasta 3 alumnos, por medio del campus virtual indicando el nombre y apellidos de todos los miembros integrantes tanto en el nombre del archivo como en el documento adjunto explicativo de la práctica.

## Diseño e Implementación

El proyecto debe ser:

- **Original.** Se pueden utilizar librerías externas para apoyar el programa general pero la implementación del algoritmo debe ser creada desde cero.
- **Documentación.** Se debe presentar un informe dónde se recoja el contexto del juego, el problema que se plantea a la IA, el algoritmo implementado, qué *controles* tiene el algoritmo, qué valor se les ha dado y cómo se maneja el programa.
- **Ejecutable.** Se debe poder ejecutar sin errores dentro de Unity3D.

### Paquete de definición del escenario

Dentro del campus virtual y de la carpeta compartida se dispondrá de un recurso que incluye un fichero de tipo **.unitypackage** en el cual se encuentra todo el esqueleto de definición de las escenas y recursos necesarios para crear la práctica

En el directorio *Scripts* del proyecto que se incluye hay un ejemplo de controlador Aleatorio <*RandomMind*> que puede valer de punto de partida para la creación de los distintos apartados del proyecto.

Si se modifica cualquier fichero de los proporcionados dentro de dicho paquete se deberá documentar convenientemente dentro de la memoria en el apartado de Decisiones de Diseño.

### Descripción detallada del entorno

Dispondremos de los siguiente elementos diferenciados dentro del escenario:

- El personaje controlado por la IA. Será representado por un avatar.
- Escenario compuesto por celdas cuadradas. Las baldosas que formarán el suelo serán de tamaño uniforme.
  - Las baldosas tendrán un coste = 1 de movimiento, salvo que se especifique lo contrario, en cuyo caso, dicho coste se simbolizará visualmente en la cuadrículas y será almacenado como una propiedad de las mismas.
- Obstáculos: Elementos que ocuparán una celda y que impedirán que sea transitable para el personaje.
- Elementos Clave: Son elementos etiquetados con un conjunto de propiedades que permitirán al personaje completar los problemas.



- Los elementos contendrán una etiqueta que los identifica.
- Enemigos: que se desplazarán por la escena de modo aleatorio.

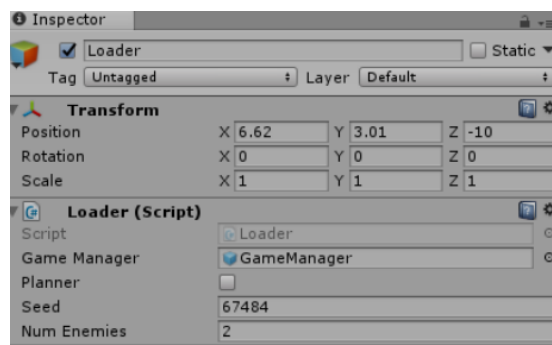


## Reglas Generales del Escenario

En el escenario, en cualquiera de los problemas, siempre dispondremos de la siguiente información y reglas fijas:

1. El tamaño del escenario (número de celdas de largo y ancho) será siempre conocido en tiempo de ejecución. Al iniciar la escena.
2. Los obstáculos son fijos dentro del escenario y el mapa no varía en ningún momento de la ejecución.
3. El agente no puede estar en una celda ocupada por un muro.
4. Se permite el movimiento en las cuatro direcciones cardinales: N-S-E-O.
5. Para activar un objeto, el jugador debe colocarse en la celda donde se encuentra el objeto.

Los objetos del escenario se generan de modo aleatorio ya sea en número o en posición a través de la semilla que se especifique en el objeto "Loader".



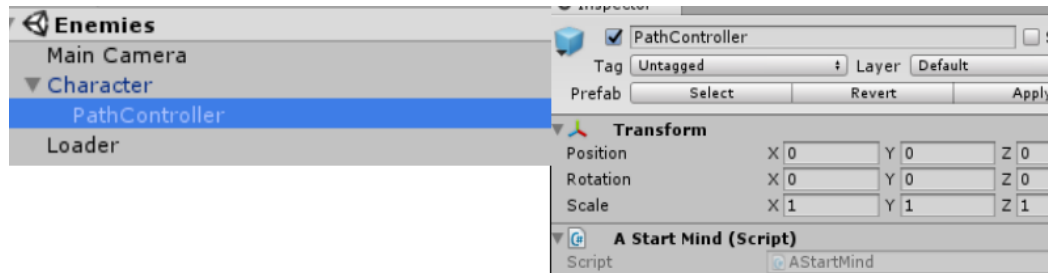
## Asignación de un Controlador

Para asignar el controlador que gestionará al personaje debemos crear un Script derivado de la Clase *AbstractPathMind* que implemente los métodos necesarios para proveer al sistema de Locomoción de los movimientos



deseados. Dicha clase se deberá implementar acompañada de todas las clases auxiliares que necesite.

Este script se asignará como componente al GameObject “*PathController*” del GameObject “*Character*”



Las clases auxiliares del personaje permiten acceder a la información del tablero en un determinado momento o a los elementos de cada celda. Los objetos del escenario están todos registrados en la clase *BoardInfo*.

## Objetivos específicos

En esta práctica se pide desarrollar dos algoritmos para el controlador en los escenarios de PathFinding y Enemies.

### Apartado 1: Búsqueda de camino de salida

En el primero deberemos crear un algoritmo de búsqueda de caminos eficiente Offline que nos encuentre la meta (Objeto etiquetado de tipo GOAL). En dicho algoritmo se emplearán las heurísticas que se consideren oportunas y se discutirán aquellos escenarios que se seleccionen (3 Semillas asignadas por grupo libremente).

Lista de semillas a probar:

- Escenario de Pathfinding: 2020, 2021, 2022 (y 2 más a elegir)

### Apartado 2: Atrapar a los enemigos

El segundo de los algoritmos a crear consistirá en una búsqueda Online que garantice que se matan a todos los enemigos dentro de la escena antes de buscar la salida. Para ellos se recomienda emplear el algoritmo anterior como base y crear un mecanismo de encontrar el enemigo más cercano para buscar el objetivo. Para matar a un enemigo basta con entrar en la celda donde se encuentra.

Lista de semillas a probar:

- Escenario de Pathfinding: 2020, 2021, 2022 (y 2 más a elegir)



## Análisis necesario

Dentro de cada uno de los apartados de la práctica se deberán realizar los siguientes análisis como mínimo:

- Tiempo medio (en milisegundos) cada vez que se llama al algoritmo en cuestión.
- Número de nodos en la lista de *abiertos* en la primera llamada que se realice al algoritmo.
- Longitud del primer camino encontrado (en el caso de enemigos, al primer enemigo elegido)
- En caso de emplear optimizaciones (eliminación de ciclos, emplear heurísticas, etc.) demostrar su influencia (mejora) en el rendimiento. Esto debe ser presentado de modo numérico, es decir, con la mejora y sin la mejora en las medidas anteriores.

## Entrega y Documentación

La entrega deberá contener un **documento** en formato PDF de diseño de la solución que deberá incluir como mínimo los siguientes elementos:

1. Portada (con el nombre de los integrantes del grupo).
2. Índice (numerado por páginas).
3. Resumen Ejecutivo: Pequeño resumen de lo que se va a entregar.
4. Descripción del algoritmo empleado para solucionar el problema y análisis.
5. Decisiones de diseño e implementación.
6. Conclusiones: Discusión sobre los resultados obtenidos.

Todas las decisiones de diseño que se hayan tomado durante el desarrollo de la práctica deberán ser explicadas detalladamente dentro de la memoria.

Asimismo, la entrega deberá incluir un **proyecto de Unity** con la escena (Scene) del proyecto, los scripts creados por el grupo, y cualquier otro elemento que sea necesario para poder hacer funcionar el proyecto.. Esto implica entrega solo y exclusivamente las carpetas *Assets*, *Packages* y *Project Settings*.

Por último, deberá incluirse **una prueba de funcionamiento** que debe ser un ejecutable para Windows (.EXE) y un vídeo de unos 2 minutos con la práctica funcionando.

Todos estos archivos se entregarán comprimidos en un fichero ZIP.

Ejemplo de la estructura de la entrega:

*Martínez-Fernández-Jorge-Gonzalez-Rodríguez-Felipe-Practica2.zip*

- Memoria.pdf
- Proyecto
  - Assets
    - Scenes
    - Scripts



- ...
- Packages
- Project Settings
- Demo
  - Demo.exe
  - ...
- Video.mp4

## Criterios de Evaluación

El proyecto se evalúa de 0 a 10.

La nota dependerá del informe generado, la implementación y la ejecución. Así mismo, la documentación de los resultados obtenidos en las distintas pruebas y cualquier comentario aclaratorio sobre las mismas.

La calificación se atenderá a los siguientes criterios de calificación:

- No ejecución de cualquier apartado o memoria altamente deficiente: Nota máxima 4
- Apartado 1 Offline
  - Desarrollo de un solo algoritmo para su resolución: 0 – 1 puntos
  - Desarrollo de dos algoritmos para su resolución: 0 – 2 puntos
  - Análisis de los resultados obtenidos: 0 - 1 punto extra
  - Documentación correcta: 0 – 1 punto extra
- Apartado 2 Online
  - Desarrollo de un solo algoritmo para su resolución: 0 – 1 puntos
  - Desarrollo de dos algoritmos para su resolución: 0 – 2 puntos
  - Análisis de los resultados obtenidos: 0 - 1 punto extra
  - Documentación correcta: 0 – 1 punto extra

Esta entrega es obligatoria para aprobar la asignatura. Se debe subir el recurso al campus en el periodo de tiempo establecido o se considerará suspensa. Se debe realizar esta entrega para que se evalúe y haga media en la convocatoria ordinaria. En caso de no presentarse la entrega o estar suspensa, se deberá mejorar y volver a realizar para convocatoria extraordinaria.